Brendan Lauterborn
Baltimore, MD ♦ 2024942334 ♦ brendan.lauterborn@gmail.com ♦ github.com/brendanglauterborn

Summary

Data Scientist with experience in machine learning, data mining, and analytics-driven decision making, specializing in predictive modeling, data quality engineering, and large-scale data analysis. Proficient in Python, R, and SQL for building end-to-end data pipelines, developing classification and regression models, and extracting actionable insights from structured and unstructured data. Experienced in applying advanced techniques including Random Forest, Gradient Boosting, XGBoost, NN, CNN, and RNN to real-world datasets. Skilled in data preprocessing, feature engineering, and model evaluation, with a strong focus on translating complex data into business impact through dashboards, reporting, and stakeholder collaboration.

Tools Utilized

Python | R | MySQL | Java | C++ | PowerShell | FastAPI | React | Docker | HuggingFace Transformers | RoBERTa | Grad-CAM | Captum | Spring Boot | RESTful APIs | PyTorch | pandas | NumPy | scikit-learn | Matplotlib | Seaborn | ggplot2 | Power BI | Microsoft Excel | Office 365 | Jupyter Notebook | RStudio | Google Colab | Postman | Git | Overleaf | OpenAI API

Education

Towson University | Master of Science - Computer Science
Texas A&M University | Bachelor of Science - Applied Mathematics

Work Experience

Graduate Teaching Assistant, - Jan 2026 to May 2026
Towson University - Department of Computer Science

- Assisted in teaching Data Structures and Object-Oriented Programming (Java), guiding students through algorithmic problem solving and computational thinking.
- Evaluated and debugged student code, identifying logical errors and improving program efficiency and correctness.
- Supported lab sessions and provided technical explanations of core concepts including recursion, data structures, and algorithm design.
- Reinforced best practices in software development, including code readability, modular design, and testing.

Business Intelligence Developer Intern - June 2025 to August 2025
CPower - Baltimore, MD

- Developed Python-based data quality pipelines to analyze and validate CRM data (Microsoft Dynamics 365), identifying ~8.5% of leads as low-quality (“Gray/Bad”) and improving overall data reliability.
- Implemented fuzzy matching algorithms to detect duplicate accounts across Dataverse and SharePoint, identifying ~250 duplicate records and improving data integrity.
- Designed data validation and transformation workflows to clean, standardize, and enrich datasets, reducing manual data cleanup efforts and improving downstream analytics accuracy.
- Performed exploratory data analysis (EDA) to uncover patterns in customer and operational data, supporting business decisions and prioritizing ~75 high-value national customers.
- Automated reporting processes using Python, enabling scalable and repeatable data analysis workflows for business stakeholders.

Projects

Humor Detection with Explainable AI (Python, PyTorch, Hugging Face, RoBERTa, FastAPI, React, Captum) GitHub

- Built and fine-tuned a RoBERTa-based humor classification model using PyTorch and Hugging Face Transformers, developing preprocessing and tokenization pipelines to transform raw user text into model-ready inputs for accurate real-time predictions
- Applied Captum Integrated Gradients to analyze token-level feature importance, improving model transparency by identifying which text tokens contributed most to classification results.
- Improved model performance by fine-tuning, increasing accuracy from 56% to 98% and improving the humor detection F1-score from 0.22 to 0.98.
- Developed a full-stack machine learning application using React and FastAPI, allowing users to submit custom text inputs, receive real-time humor predictions, and visualize Integrated Gradients token-level attribution scores explaining model decisions.

Brain Tumor Classification with Explainable AI (Python, PyTorch, CNNs, Transfer Learning, Grad-CAM) GitHub

- Built and trained a custom convolutional neural network (CNN) for brain tumor classification using PyTorch, implementing image preprocessing pipelines to transform MRI scans into model-ready inputs for tumor detection.
- Improved model performance by applying transfer learning with DenseNet-121, increasing accuracy from 79% to 88% and improving meningioma tumor F1-score from 0.58 to 0.77 compared to the custom CNN baseline.
- Applied Grad-CAM Explainable AI techniques to generate class activation heatmaps, visualizing MRI regions that contributed most to tumor classification decisions and improving model interpretability.

CNN-Based Computer Vision System for Dog Breed Classification and Human–Dog Mapping (Python, PyTorch) GitHub

- Developed a computer vision application that detects whether an image contains a human or a dog, classifies dog breeds across 133 categories, and maps human faces to visually similar dog breeds; improved model accuracy from ~13% to ~86% by replacing a baseline CNN with DenseNet-161 transfer learning.
- Enabled interactive and user-facing functionality by supporting real-time image classification and human-to-breed matching across a large multi-class dataset.

Subreddit Sentiment Classifier (roBERTa, Python)

- Fine-tuned a RoBERTa transformer model to classify Reddit comments into quality categories (Controversial, Baseline, High-Quality, Viral).
- Developed an NLP workflow including preprocessing, transformer tokenization, and supervised training, achieving F1 ≈ 0.79 on validation data.

Natural Language to SQL – LLM Prompt Engineering (Python, OpenAI API)

- Developed a benchmark and automated Python evaluation framework to measure how reliably LLMs translate natural language into executable SQL queries.
- Ran controlled prompt-engineering experiments (zero-shot, few-shot, chain-of-thought) to quantify accuracy tradeoffs, identify failure modes, and determine strategies that improve reliability of LLM-generated queries.
- Produced actionable guidance on prompting strategies that improve trustworthiness of LLM-driven analytics workflows.

MovieLens Recommender System (Python) GitHub

- Built collaborative filtering and ALS matrix factorization recommenders to generate personalized movie suggestions from user behavior data.
- Quantified improvements using ranking metrics (Precision@K, Recall@K, MAP, NDCG), showing superior recommendation relevance compared to baseline similarity models.
- Demonstrated how latent-factor recommendation systems drive user engagement and retention through personalized content discovery.

Academic Papers | View Papers